

The Complete Modern API Exception Handling Guide (.NET 8/9)

In modern .NET (version 8 and newer), the framework has unified how APIs handle errors. You no longer need custom middleware or exception filters. A complete, production-ready setup relies on three interconnected pillars:

Component	Purpose	What it catches
1. ProblemDetails	The Formatter (RFC 7807)	Formats all error responses into a standard JSON structure.
2. ExceptionHandler	The Interceptor	Catches actual C# crashes (e.g., <code>SqlException</code> , <code>NullReferenceException</code>).
3. Status Code Pages	The Gap-Filler	Catches non-crashing HTTP errors (e.g., 404 Route Not Found, 401 Unauthorized).

1. The Problem Details Engine (The Foundation)

Every error in a modern API must return RFC 7807 JSON. To do this, you register the engine in `Program.cs`. You can also enrich every error with standard metadata, like a Trace ID for your observability stack (Serilog/Datadog).

```
// In Program.cs
builder.Services.AddProblemDetails(options =>
{
    options.CustomizeProblemDetails = ctx =>
    {
        ctx.ProblemDetails.Instance = $"{ctx.HttpContext.Request.Method}
{ctx.HttpContext.Request.Path}";
        ctx.ProblemDetails.Extensions.TryAdd("traceId",
ctx.HttpContext.TraceIdentifier);
    };
});
```

2. Handling Non-Exceptions (The Missing Piece)

If a user sends a JWT that is expired, the authentication middleware rejects it with a **401 Unauthorized**. Because no exception was thrown, `ExceptionHandler` is entirely bypassed! To ensure these return Problem Details JSON, you must enable Status Code Pages.

```
// In Program.cs Pipeline (before app.MapControllers)
app.UseStatusCodePages(); // Automatically translates empty 400-599 responses
into Problem Details
```

3. The Chain of Responsibility (`ExceptionHandler`)

Modern .NET uses a chain of handlers. You can write small, focused handlers rather than one massive try-catch block.

A. The Validation Handler

This specifically catches `FluentValidation` errors and formats them into a `ValidationProblemDetails` object (which includes a dictionary of field-specific errors).

```

public class ValidationExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext context, Exception
exception, CancellationToken token)
    {
        if (exception is not ValidationException validationException)
            return false; // Not my job, pass to the next handler!

        var problemDetails = new
HttpValidationProblemDetails(validationException.Errors
        .GroupBy(e => e.PropertyName, e => e.ErrorMessage)
        .ToDictionary(g => g.Key, g => g.ToArray()))
        {
            Status = StatusCodes.Status400BadRequest,
            Title = "Validation failed"
        };

        context.Response.StatusCode = StatusCodes.Status400BadRequest;
        await context.Response.WriteAsJsonAsync(problemDetails, token);
        return true; // Handled! Stop the chain.
    }
}

```

B. The Global Fallback Handler

This catches everything else (Database timeouts, Null References, etc.) and acts as the final safety net.

```

public class GlobalExceptionHandler(IProblemDetailsService
problemDetailsService, ILogger<GlobalExceptionHandler> logger) :
ExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext context, Exception
exception, CancellationToken token)
    {
        // 1. Log the severe error
        logger.LogError(exception, "Unhandled exception: {Message}",
exception.Message);

        // 2. Set Status Code
        context.Response.StatusCode =
StatusCodes.Status500InternalServerError;

        // 3. Hand off to ProblemDetailsService to output the RFC 7807 JSON
        return await problemDetailsService.TryWriteAsync(new
ProblemDetailsContext
        {
            HttpContext = context,
            Exception = exception,
            ProblemDetails = new ProblemDetails
            {
                Title = "An unexpected server error occurred.",
                Status = StatusCodes.Status500InternalServerError
            }
        });
    }
}

```

4. The Perfect Program.cs Pipeline

The order of registration dictates the order of execution. Registration is the most critical part of the modern setup.

```
// -----
// 1. BUILDER PHASE (Services)
// -----
builder.Services.AddProblemDetails(); // Add the Formatter engine

// Add handlers in the exact order you want them checked
builder.Services.AddExceptionHandler<ValidationExceptionHandler>();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();

var app = builder.Build();

// -----
// 2. APP PHASE (Middleware Pipeline)
// -----
app.UseExceptionHandler(); // 1st: Catches actual Exceptions
app.UseStatusCodePages(); // 2nd: Formats empty errors (401, 404) into
// Problem Details

app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Important Rule: Do not use `UseDeveloperExceptionPage()` in combination with this setup if you want to test how your API returns JSON. The Developer page intercepts the error and returns HTML before your `IExceptionHandler` ever gets a chance to run.